Taller ESP32 WROOM-32 + PlatformIO + Wokwi

GPIO, UART, PWM, ADC, I2C, LCD e Interrupciones

Esteban Isidro Acero

Pontificia Universidad Católica del perú

20 de marzo de 2026

Agenda general (3 días)

- **Día 1**: Introducción al ESP32, PlatformIO, estructura de programa, GPIO y LEDs.
- **Día 2**: UART, PWM y ADC. Práctica: mezcla RGB con 3 potenciómetros.
- **Día 3**: I2C, LCD 16x2, interrupciones. Mini-proyecto integrador.

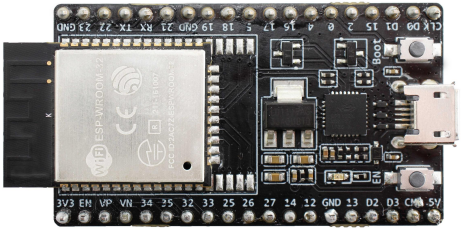
Día 1: Objetivos

Al finalizar, el alumno podrá:

- Configurar **PlatformIO** y cargar un proyecto al ESP32.
- Crear y simular proyectos mediante **Wokwi**
- Entender la estructura de un programa Arduino: **setup()** y **loop()**.
- Controlar **GPIO** como salida digital.
- Implementar patrones básicos con LEDs (blink, secuencias).
- Implementar interfaces básicas de comunición (bus serial, Pantalla LED).
- Hacer uso de interrupciones para manejar interacciones con los periféricos y consola.

¿Qué es el ESP32 WROOM-32?

- Microcontrolador con CPU, memoria y periféricos integrados.
- Alimentación lógica típica: **3.3V** (cuidado con módulos 5V).
- GPIO configurables como entradas/salidas (con algunas restricciones).
- Usaremos: GPIO, UART (Serial), PWM, ADC, I2C.



Recordatorio: GPIOs a 3.3V (no toleran 5V).

Entorno de desarrollo: PlatformIO

Flujo típico (VS Code)

- Instala **VS Code** y la extensión **PlatformIO**.
- **New Project**:
  - Board: *ESP32 Dev Module* (`board = esp32dev`).
  - Framework recomendado para clase: **Arduino**.
- Código en `src/main.cpp` (no en `.ino`).
- Configuración en `platformio.ini`.

Estructura del proyecto

```
project/
 platformio.ini
 src/main.cpp
 lib/   (opcional)
```

platformio.ini

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
; Opcional
monitor_speed = 115200
; (115200/460800)
upload_speed = 921600
; Ejemplos de librerias
; lib_deps =
;    madhephaestus/ESP32Servo@^1.
```

Estructura básica de un programa

- **setup()**: corre una sola vez (configuración).
- **loop()**: corre en bucle (lógica principal).

```
#include <Arduino.h>

void setup() {
Serial.begin(115200);
}

void loop() {
// c digo repetitivo
}
```

Botones y comandos útiles

- **Build**: compila el firmware.
- **Upload**: flashea por USB.
- **Monitor**: ve `Serial` en tiempo real.

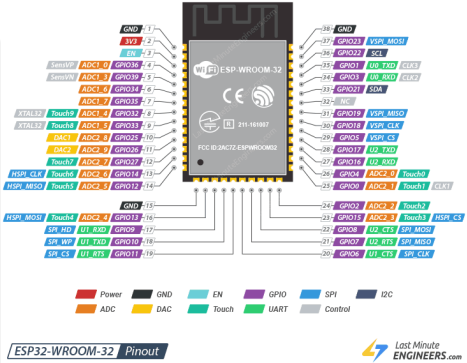
Si el Upload falla (BOOT/EN)

- Mantén presionado **BOOT** cuando quede en *"Connecting..."*.
- Suelta BOOT cuando empiece a escribir.
- **EN** reinicia la placa.
- Revisa: cable USB de datos, driver USB-UART (CP210x/CH340), puerto correcto.

Disposición de pines ESP32

Reglas prácticas (DevKit / WROOM-32)

- **Evita** GPIO 6–11: conectados a la **flash** del módulo.
- **Solo entrada**: GPIO 34–39 (ideal para sensores analógicos/digitales).
- **Strapping (arranque)**: GPIO 0, 2, 4, 5, 12, 15. Úsalos con cuidado.
- **ADC + Wi-Fi**: prefiere **ADC1** (GPIO 32–39).
- **PWM**: sirve para LED/motores/servos (en GPIO "seguros").
- I2C típico: SDA=21, SCL=22 (configurable).



Pinout de referencia (DevKit).

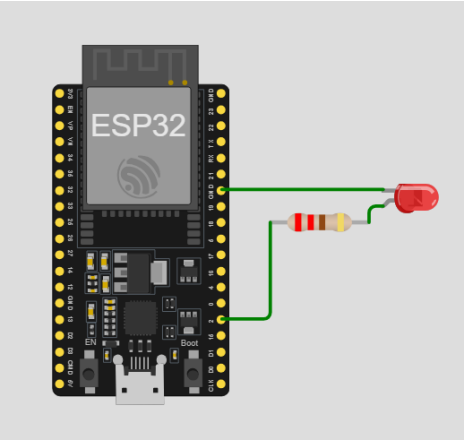
Práctica: Blink (LED)

```
#include <Arduino.h>

const int LED_PIN = 2;

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(300);
  digitalWrite(LED_PIN, LOW);
  delay(300);
}
```

- **Salida**: controla un pin en HIGH/LOW.
- Funciones: **pinMode()**, **digitalWrite()**, **delay()**, **serial.begin()**, **serial.println()**, **serial.read()**.
- Recomendación: usar resistencia (220–330Ω) en LEDs.

Práctica: Blink (LED)

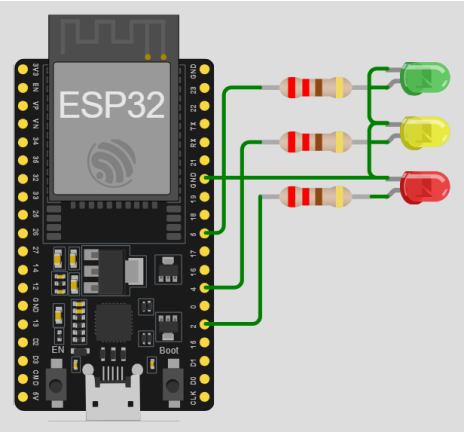
- Objetivo: aplicar lo aprendido para crear un proyecto básico desde cero y/o simularlo.
- Buenas prácticas:
  - Definir pines como constantes.
  - Usar funciones para evitar código repetido.



Pin config.

Checklist Día 1

- PlatformIO instalado y reconoce el puerto del ESP32.
- Compilo y subo un proyecto sin errores.
- LED enciende/apaga con Blink.
- Secuencia de LEDs.
- Creación y simulación de proyectos con Wokwi



Ejercicio práctico.

Día 2: Objetivos

- Usar **UART** (Serial) para depuración y comandos.
- Aplicar **PWM** para controlar brillo (duty cycle).
- Leer **ADC** con potenciómetros.
- Integrar: **3 potenciómetros** $\rightarrow$ **PWM RGB**.

UART (Serial)

- UART: TX/RX.
- Baud rate: tasa de transmisión de **símbolos**.
- Comunicación **Serial**: (USB) para monitor.
- Funciones: **Serial.begin()**, **Serial.println()**, **Serial.available()**, **Serial.read()**.

```
Serial.begin(115200);
Serial.println("Hola");
if (Serial.available()) {
  char c = Serial.read();
}
```

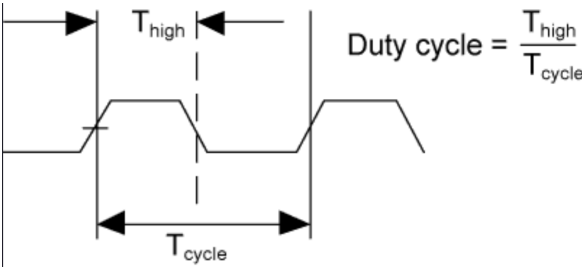
Confirmación básica de la comunicación.

PWM en ESP32

- PWM = señal digital con **duty cycle** variable.
- Resolución 8 bits: duty 0–255.
- Frecuencia: $\leq 40MHz$.

$$V_{\text{prom}} = dc\, V_h + (1 - dc)\, V_l$$

$$V_{\text{prom}} = V_l + dc\,(V_h - V_l)$$



Configuración PWM en ESP32

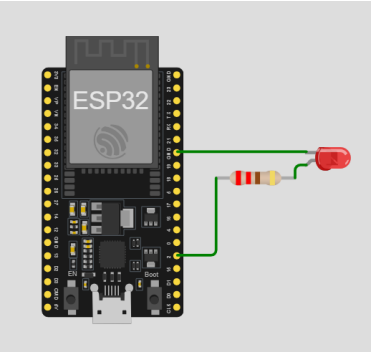
Dependiendo de la versión de tu core:

```
ledcSetup(PWM_CHANNEL, PWM_FREQ_HZ, PWM_RES_BITS);
ledcAttachPin(LED_PIN, PWM_CHANNEL);
ledcWrite(PWM_CHANNEL, PWM_DUTY);
```

Hasta 2.0.

```
ledcAttach(LED_PIN, PWM_FREQ_HZ, PWM_RES_BITS);
ledcWrite(LED_PIN, PWM_DUTY);
```

De 3.0 en adelante.



Ejercicio práctico.

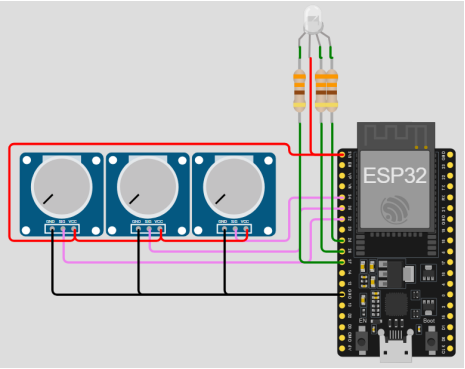
ADC en ESP32

- ADC convierte voltaje a número (típico 0–4095 con 12 bits).
- Potenciómetro: extremos a 3.3V y GND, cursor a pin ADC.

```
analogReadResolution(12);
int v = analogRead(34); // 0..4095
```

Conexiones sugeridas

Conexión	GPIO
Pot R (ADC)	**GPIO 34**
Pot G (ADC)	**GPIO 35**
Pot B (ADC)	**GPIO 32**
LED R (PWM)	**GPIO 25**
LED G (PWM)	**GPIO 26**
LED B (PWM)	**GPIO 27**



Ejercicio práctico.

Práctica: mezcla RGB (ADC → PWM)

```
#include <Arduino.h>

const int POT_R = 34, POT_G = 35, POT_B = 32;
const int LED_R = 25, LED_G = 26, LED_B = 27;

const int CH_R = 0, CH_G = 1, CH_B = 2;
const int FREQ = 5000, RES = 8; // duty 0..255

int adcToPwm(int x) { return map(constrain(x,0,4095), 0,4095, 0,255); }

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);
  ledcSetup(CH_R, FREQ, RES); ledcAttachPin(LED_R, CH_R);
  ledcSetup(CH_G, FREQ, RES); ledcAttachPin(LED_G, CH_G);
  ledcSetup(CH_B, FREQ, RES); ledcAttachPin(LED_B, CH_B);
}
```

Práctica: mezcla RGB (ADC → PWM)

```
void loop() {
  int r = adcToPwm(analogRead(POT_R));
  int g = adcToPwm(analogRead(POT_G));
  int b = adcToPwm(analogRead(POT_B));

  ledcWrite(CH_R, r);
  ledcWrite(CH_G, g);
  ledcWrite(CH_B, b);

  delay(50);
}
```

Checklist Día 2

- Serial Monitor muestra valores ADC (opcional).
- Cada potenciómetro controla un canal RGB.
- Se observa mezcla de colores estable.
- INVESTIGAR: suavizado por promedio móvil.

Día 3: Objetivos

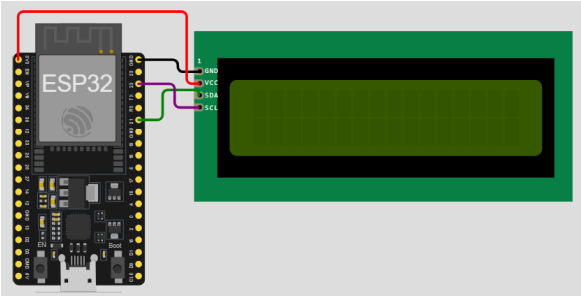
- Entender el bus **I2C** (SDA/SCL, dirección).
- Mostrar información en LCD 16x2 por I2C.
- Usar **interrupciones** para un botón (respuesta inmediata).
- Integrar en mini-proyecto: botón cambia pantalla / modo.

platformio.ini (Día 3 con LCD I2C)

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200

lib_deps =
  LiquidCrystal_I2C@^1.1.4
```

I2C: conceptos rápidos

- Bus compartido: **SDA** (datos), **SCL** (reloj).
- Dispositivos identificados por **dirección** (ej: 0x27).
- En ESP32 DevKit: común usar **GPIO 21** como SDA y **GPIO 22** como SCL.
- Librería típica en PlatformIO: `LiquidCrystal_I2C`.
- Funciones: **init()**, **backlight()**, **setCursor()**, **print()**, **clear()**.



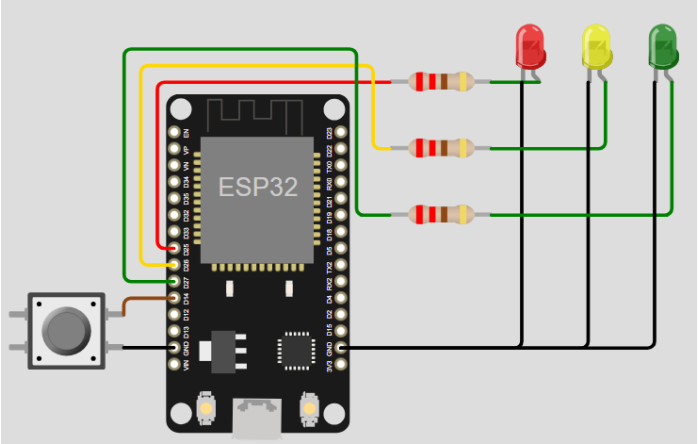
Ejemplo "Hello world".

Configuración básica

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
//LiquidCrystal_I2C nombre_objeto(direccion_I2C, columnas, filas);

void setup() {
  Serial.begin(115200);
  // SDA = GPIO 21
  // SCL = GPIO 22
  Wire.begin(21, 22);
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Hello World!");
  lcd.setCursor(0, 1);
  lcd.print("Desde: ESP32");
}
```

Interrupciones

- Evita *polling* constante del botón.
- ISR debe ser **corta** (no LCD dentro de ISR).
- Variables compartidas: usar **volatile** y flags.
- Anti-rebote: filtro por tiempo con **millis()**.

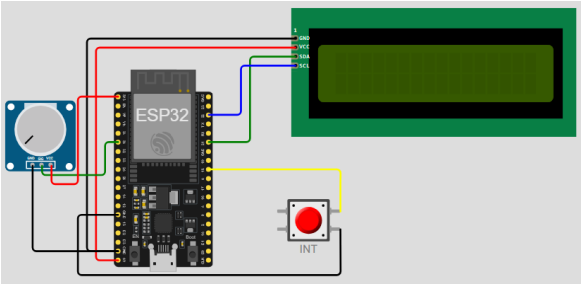


Ejercicio práctico.

Proyecto

Dashboard LCD con botón controlado por interrupciones

- Modo 0: mensaje inicial + contador.
- Modo 1: mostrar tiempo activo (segundos).
- Modo 2: mostrar el valor del potenciómetro.



Ejercicio práctico.

Checklist Día 3

- LCD muestra texto correctamente (0x27).
- Botón cambia de modo usando interrupción.
- Anti-rebote funciona (sin múltiple conteo).
- Mini-proyecto integrado y estable.

Cierre

- Día 1: GPIO + LEDs + PlatformIO.
- Día 2: UART + PWM + ADC (RGB con potenciómetros).
- Día 3: I2C + LCD + Interrupciones (mini-proyecto).

Siguiente paso (opcional): WiFi/BLE, sensores I2C, logging, FreeRTOS.